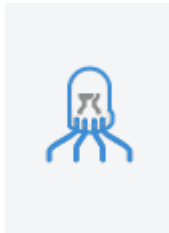


Proximity Alarm Project

In this project, we'll combine components that use **sound**, **light**, and **motion** together. The circuit will be a simple **object detector**, which gives feedback about how close an object is to the circuit through the different included components.

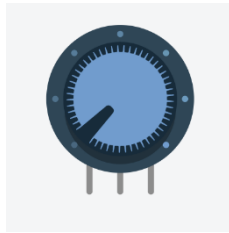
Materials:



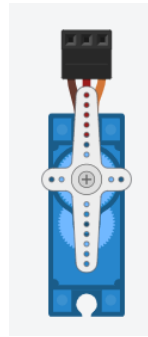
1 x RGB LED



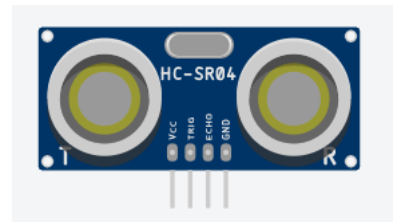
Resistor:
1 x 220Ω



1 x Variable Resistor



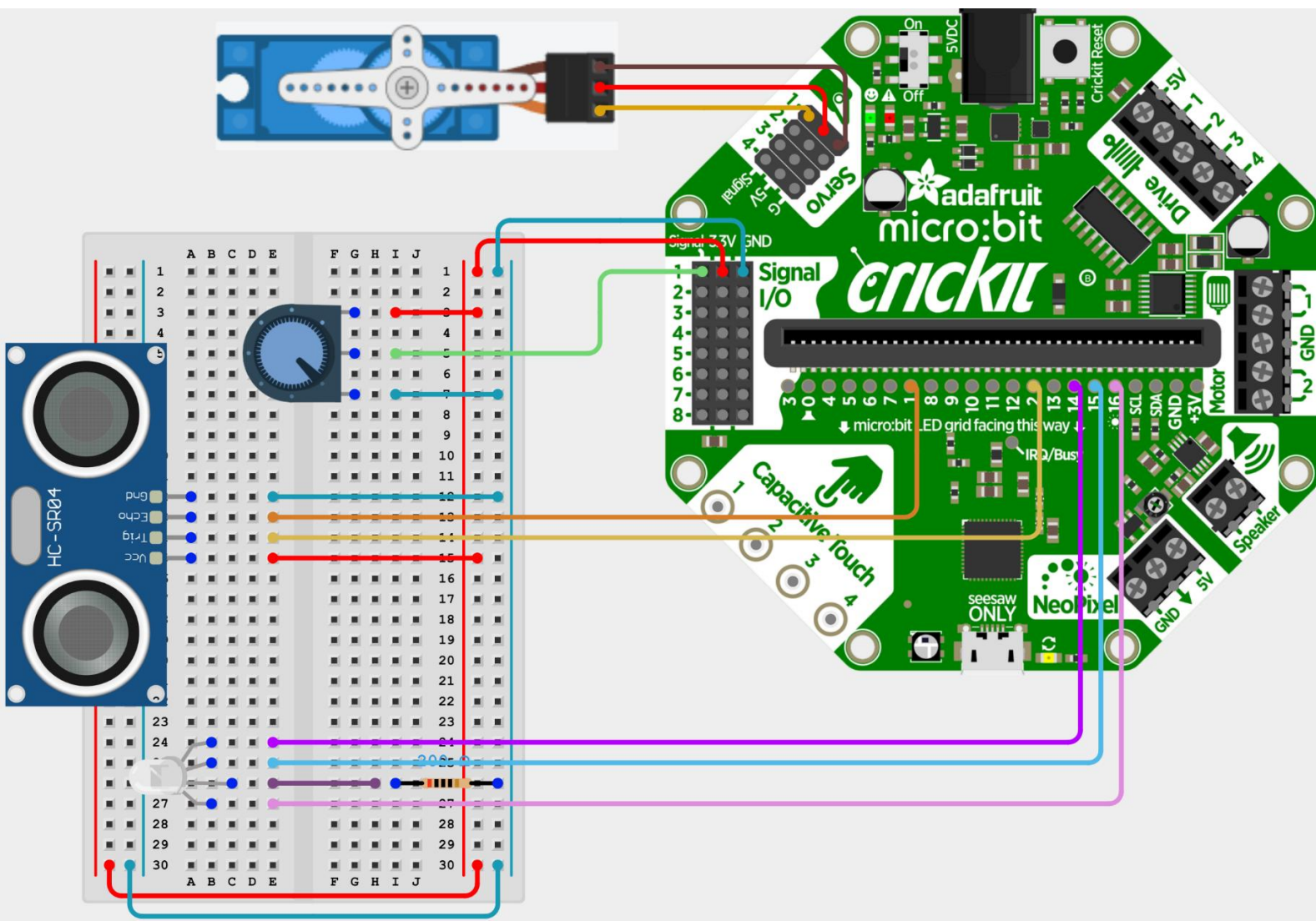
1 x Servo



1 x Ultrasonic Sensor

Wiring Diagram:

Wire your project according to the following schematic.



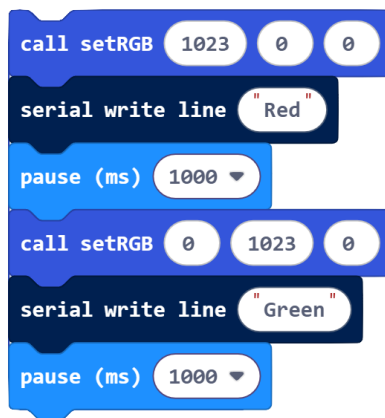
Verify Circuit

As the complexity of this circuit is increasing, so is the possibility of errors in component placement (or poor connections for the physical circuit). Before getting into the complexity of the whole project, it would be a good idea to **verify that you've wired the components correctly**, by individually running a simple test on each.

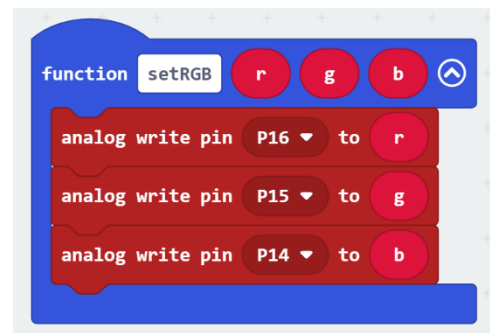
Note: *Once you're done a test, disable that code and start on the next test. You shouldn't need to run all the tests concurrently.*

Test One: RGB LED

We have 3 separate pins to control the individual R, G, and B LEDs within the single RGB LED. Try turning each individual LED on one at a time to verify the circuit is correct:



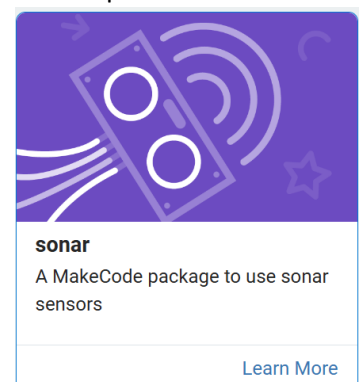
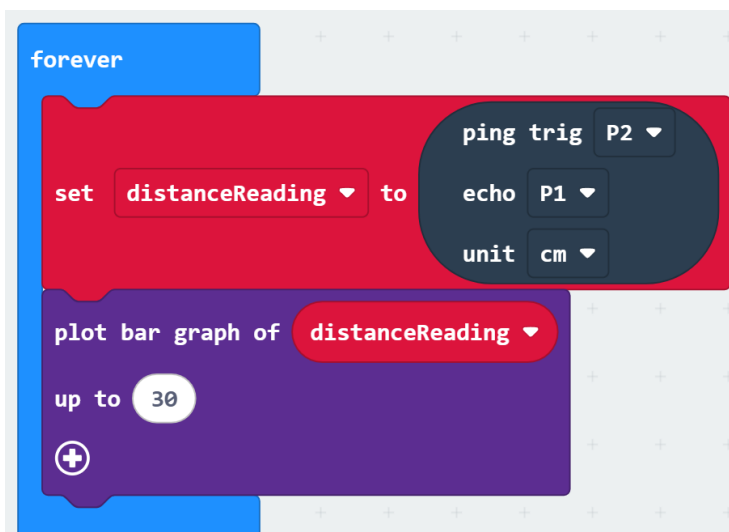
...Add similar blocks to test **blue**



Test Two: Ultrasonic Sensor

Our ultrasonic sensor should be able to report how far away an object is (if there is one at all!). Use a **forever loop** to take readings and graph them on the LED grid.

- Don't forget to power the Crickit Board; this sensor won't work without that 5v power source.

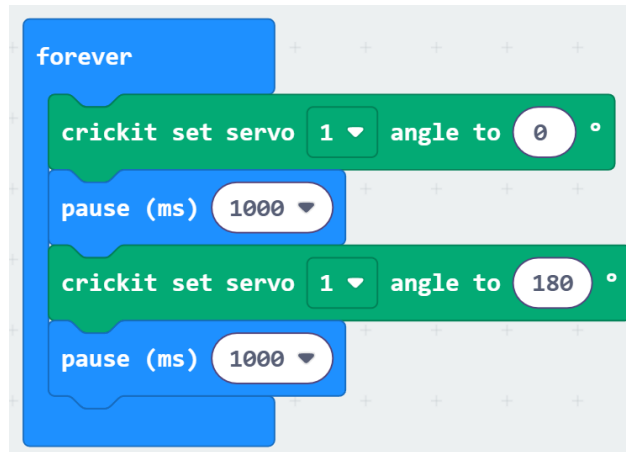


Add **sonar** extension to drive this component.

Test Three: Servo Motor

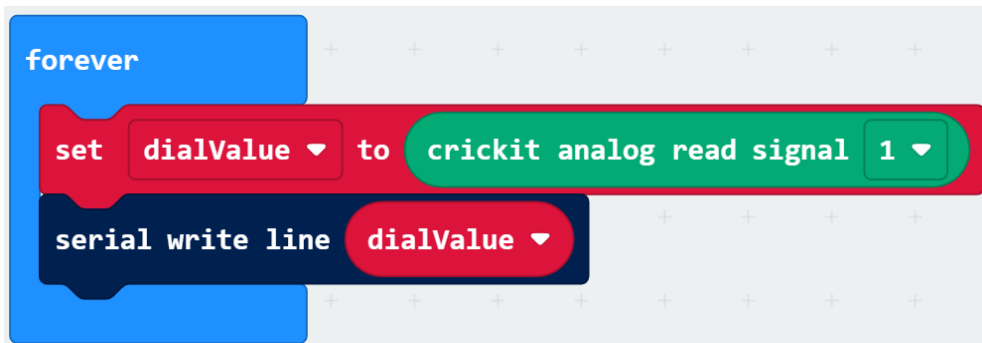
To ensure your servo is wired correctly and in good repair, have it move across its range of motion. Remember to include a wait to allow the Servo to reach its target angle.

- **NOTE:** using angle measurements (0 – 180 degrees) will often result in less movement being observed (roughly 90 degrees total).
- This is because the servo angle block maps to a particular PWM range (1000 – 2000, likely). However, not all servos use that as their full range.
- If you observe a reduced angle range, try instructing the servo to move between PWM range of 500 – 2500; this may fix the issue.



Test Four: Potentiometer

Changing the dial position should give us a different reading. Simplest way to verify is to take a reading and immediately print it to the Serial Monitor:



Assignment Details: Base Features

For this assignment, completing the following base features will earn you a maximum mark of 9/10. Some hints and explanation will be given, but you'll need to construct a code solution on your own.

It may be very useful to look back at your previous demos as reference material.

Start with distance:

The actions taken by your circuit should be entirely dependent on what the **Ultrasonic Sensor**. Begin your code by **polling the Ultrasonic Sensor**, and storing its result in a **variable**.

distanceReading ▼

If you've completed all the component tests above, this step should already be done!

3 Alarm Levels:

Depending on the object distance detected by the **Ultrasonic Sensor**, one of three "Alarm Levels" should be enacted. What that means for each of the components (LED, Speaker, Servo) is listed below.

distance $\geq$ 30 (cm) or distance == 0: All Clear

- RGB LED: Turn to Green
- Speaker: No Noise
- Servo: No Motion

Recall that distance == 0 implies "**no object**"

distance < 30 (cm) and distance $\geq$ 15 (cm): Warning

- RGB LED: Turn to Yellow
- Speaker: One Tone, Low Pitch
- Servo: Small Movement (back and forth between 70° and 110°)

distance < 15 (cm) and distance > 0 (cm): Alarm

- RGB LED: Turn to Red
- Speaker: 2+ Tones alternating, Different pitches
- Servo: Large Movement (back and forth between 10° and 170°)

Volume Control:

Sometimes the speaker can seem a bit loud. Turn your potentiometer into a *code-driven* volume knob.

- Map the full rotation of the potentiometer to volume levels:
 - Far left: volume **0**
 - Far right: volume **255**
- All positions in between should adjust the volume proportionally.

Assignment Details: Challenge Feature

Complete the following *Challenge Feature* to earn the last 1/10 for the project:

Multi-purpose Dial

Modify your code so that the dial position will also *control the brightness of the RGB LED*.

- The user should be able to move the dial to dim the LED to different amounts
- Correct colors should be maintained (*more or less, at least...*)

----- Suggestions and Hints are below but try to build what you can on your own first! -----

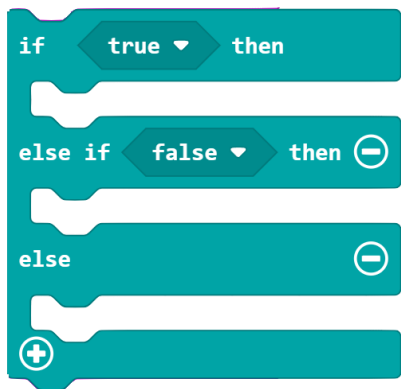
Hints and Suggestions:

Start with distance:

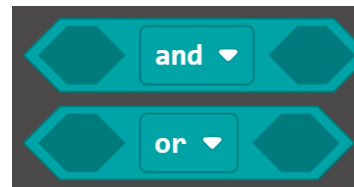
This feature is easy! In fact, the section on **testing the Ultrasonic Sensor** above provides the exact script we'd need to poll the sensor and store the reading in a variable.

3 Alarm Levels:

The description of this feature shows 3 *distinct responses* based on the Ultrasonic reading. You'll want to use an **IF Statement that had 3 branches**:



Notice how the main description of the 3 alarm levels use **and** and **or** to define the associated distance. You can find these blocks under [Logic](#)

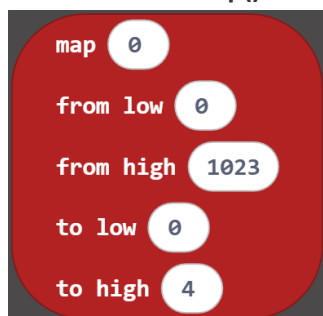


Volume Control

If you look at the  block, you should notice it expects a value in the range of **0 ... 255**

The potentiometer's reading should be value in the **0 ... 1023** range. Before you can use your potentiometer reading to impact the volume, you must first *remap the value onto the correct number range*.

Use one of the **map()** blocks to accomplish this:



Multipurpose Dial

Try solving this for **red** and **green** first. These are simpler since no LED color mixing is required to achieve these hues.

Recall that  expects values in the range of **0 ... 1023**

And as above, the potentiometer's reading should fall within the same range...how convenient!

When mixing for yellow, if your base *unfaded* color was using a value less than **1023** in the `analogWrite()` instruction, then it may be necessary to use **`map()`** to remap your potentiometer value onto the range **0 ... normalMaxValue**